Le clustering

Trouver des groupes que personne n'a nommés

Joseph Azar

Maître de conférences
Université Marie et Louis Pasteur
ELC2 · Introduction au machine learning · Séance 3

Plan

1. Chercher des groupes quand il n'y a pas de réponse
2. K-Means, pas à pas sur 9 clients
3. Standardiser, puis choisir le nombre de groupes
4. Lire les groupes d'un vrai fichier de 600 clients
5. Trois autres outils en un coup d'œil, puis le code Python

🎉 Une fête avec des inconnus

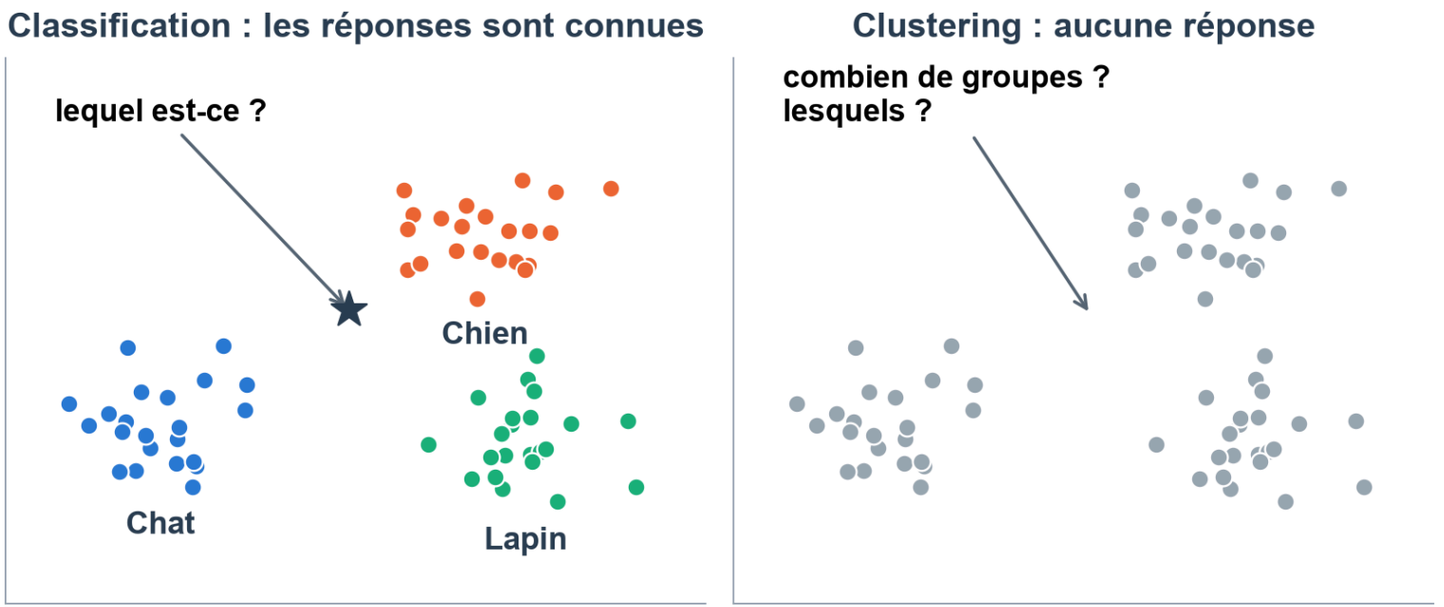
Vous entrez dans une salle : 100 personnes que vous ne connaissez pas.

Sans que personne ne vous explique rien, vous voyez des **groupes** :

- des gens en costume, qui parlent travail ;
- des gens en jeans, qui parlent examens ;
- des gens très élégants, près du buffet.

Vous venez de faire du **clustering** : **regrouper ce qui se ressemble**, sans qu'on vous ait donné les groupes à l'avance.

Classer ou regrouper ?



Classification (séance 2) : on a des exemples **avec leur réponse.** On apprend à deviner la réponse d'un nouveau cas.

Clustering : **aucune réponse.** On cherche des groupes dans les données. On dit « non supervisé ».

🌍 Où l'utilise-t-on ?

Marketing

- Découper la clientèle en profils
- Une offre par profil

Technologie

- Ranger des documents, des photos
- Repérer le comportement anormal

Santé

- Groupes de patients proches
- Sous-types d'une maladie

Réseaux sociaux

- Communautés d'utilisateurs
- Sujets du moment

🛒 Le projet : Comtoise Market

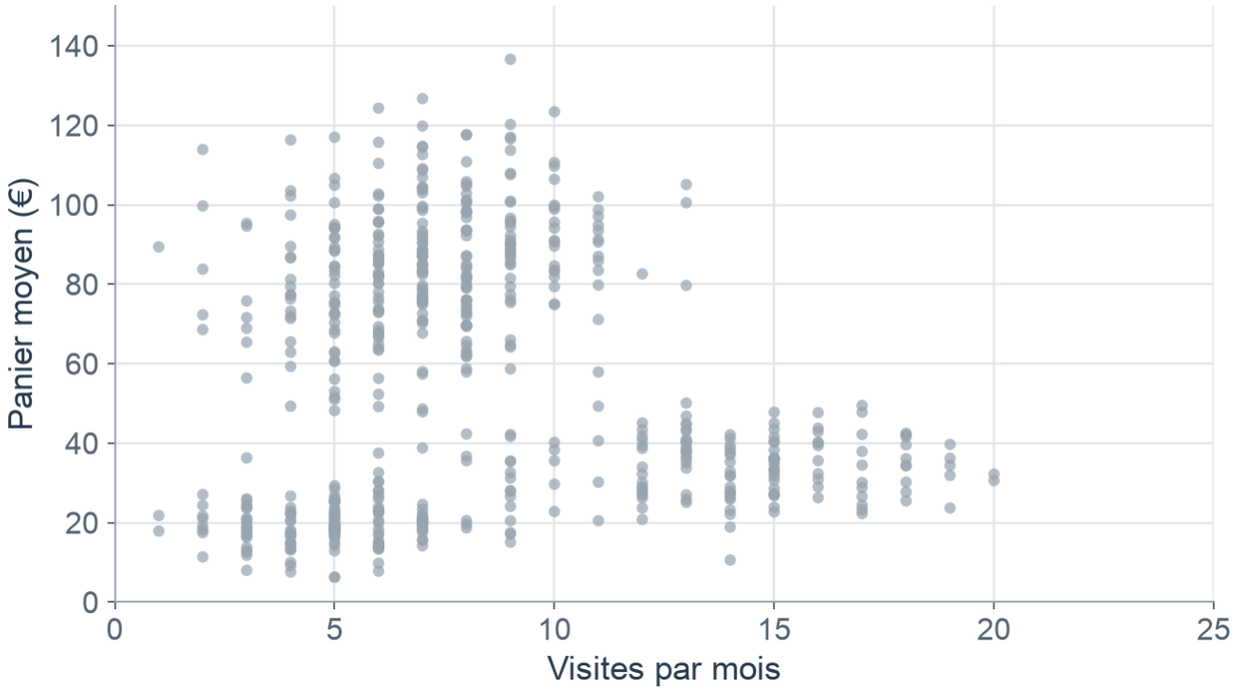
Un supermarché de Franche-Comté a **600 clients** avec une carte de fidélité. La direction veut **des offres différentes selon les types de clients**, mais personne n'a jamais rangé les clients dans des catégories.

Client	Âge	Visites par mois	Panier moyen (€)	Part en promo (%)	Part de bio (%)
M-0001	66	12	28,7	17	19
M-0002	39	10	91,0	38	8
M-0003	26	5	26,7	56	11
M-0004	68	8	42,3	17	29
M-0005	55	14	37,1	20	9
M-0006	21	2	17,5	59	5

extrait de clients-comtoise-market.csv · données fictives, fichier propre (séance 1 faite)

Pas de colonne « réponse » ici. Aucune cible à deviner : on cherche seulement **qui ressemble à qui**.

👀 Regardons les clients

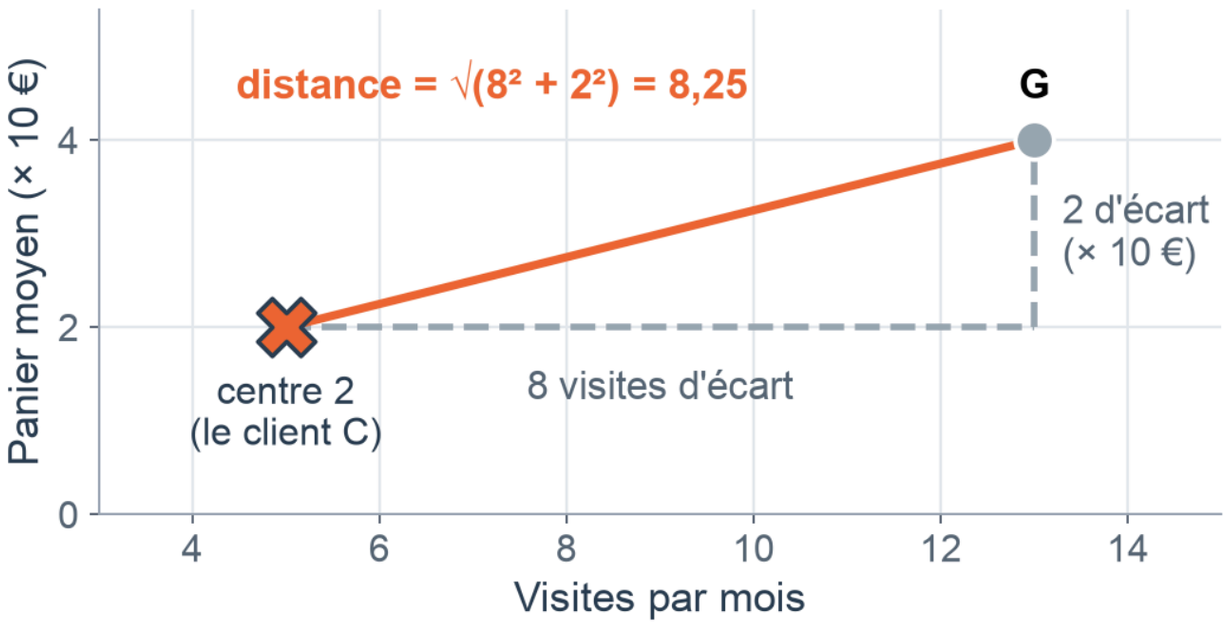


Deux colonnes sur cinq : **visites par mois** et **panier moyen.**

À l'œil, on voit des **amas.**

Mais avec cinq colonnes, l'œil ne suffit plus. On demande à l'ordinateur de chercher les groupes **à notre place.**

Se ressembler, c'est être proche



Deux clients se ressemblent si leurs chiffres sont **proches** : leur **distance** est petite.

$$\text{distance} = \sqrt{(\text{écart}_1^2 + \text{écart}_2^2)}$$

D'où vient-elle ? De Pythagore : les deux écarts forment les côtés d'un triangle rectangle, la distance est la ligne droite qui les relie. Ici : 8 visites et 2 de panier, $\sqrt{(64 + 4)} = \sqrt{68} = 8{,}25$. (Même calcul qu'en séance 1.)

📊 K-Means : l'idée en quatre gestes

K = le nombre de groupes qu'on demande. À nous de le choisir.

1. **Choisir** K points au hasard : les **centres**.

2. **Rattacher** chaque client au centre le plus proche.

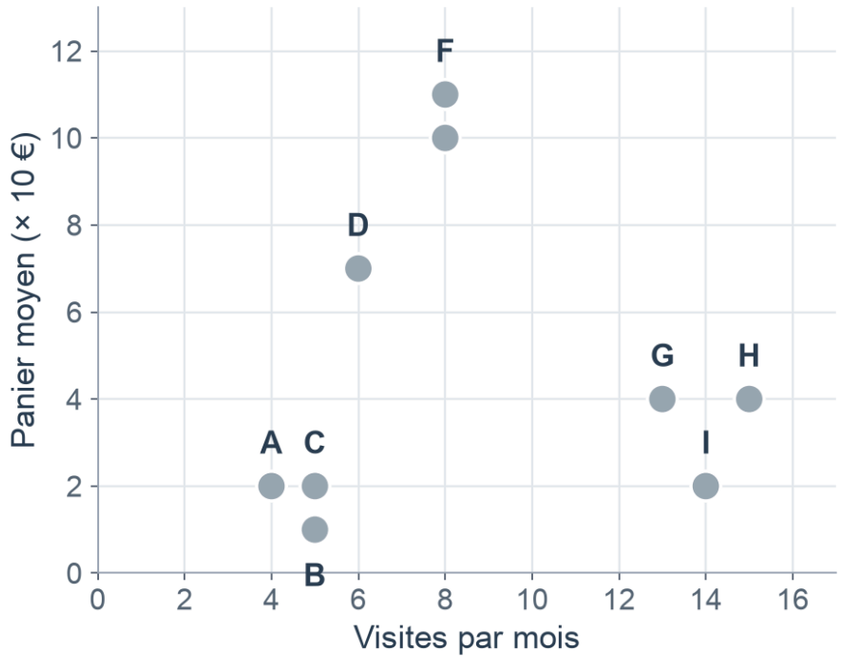
3. **Déplacer** chaque centre au milieu (la moyenne) de son groupe.

4. **Recommencer** les gestes 2 et 3, jusqu'à ce que plus rien ne bouge.

K-Means sur 9 clients

0 Les clients | 1 Les centres | 2 Rattacher | 3 Déplacer | 2 Rattacher | 3 Déplacer | Fini

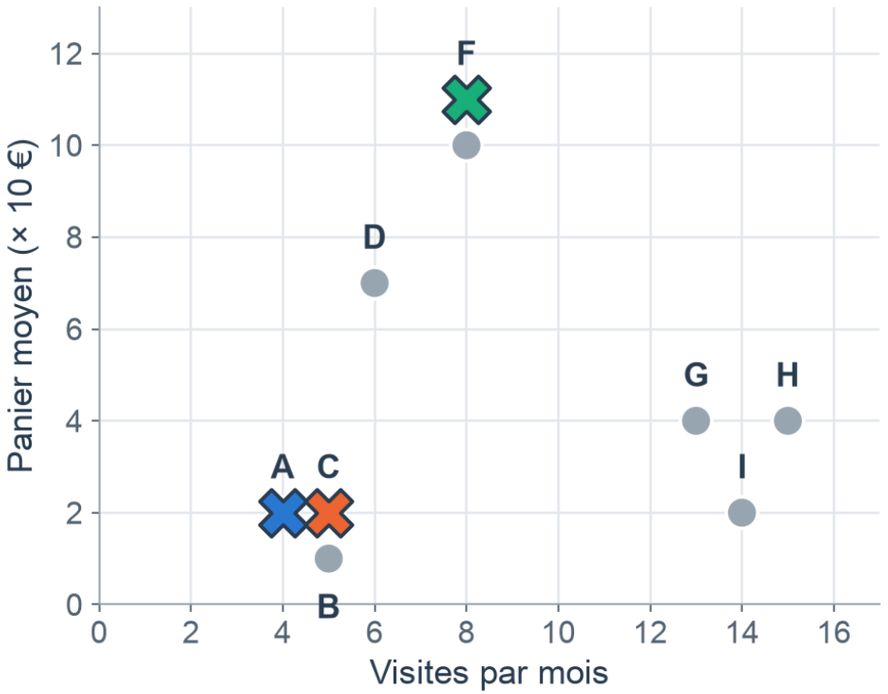
Client	Visites par mois	Panier (× 10 €)
A	4	2
B	5	1
C	5	2
D	6	7
E	8	10
F	8	11
G	13	4
H	15	4
I	14	2



chiffres arrondis, les deux axes à la même échelle, pour calculer à la main · K = 3

Geste 1 : trois centres au hasard

0 Les clients | **1 Les centres** | 2 Rattacher | 3 Déplacer | 2 Rattacher | 3 Déplacer | Fini



Le sort désigne trois clients comme centres de départ :

- centre 1 = **A** (4 ; 2)
- centre 2 = **C** (5 ; 2)
- centre 3 = **F** (8 ; 11)

Un tirage **volontairement mauvais** : deux centres côte à côte. L'algorithme va devoir se corriger.

🧮 Geste 2 : où va le client G ?

0 Les clients | 1 Les centres | **2 Rattacher** | 3 Déplacer | 2 Rattacher | 3 Déplacer | Fini

Ce qui entre

G (13 ; 4) et les trois centres.

→

Ce qu'on fait

La distance de G à chaque centre.

→

Ce qui sort

G rejoint le **centre 2**.

- G à A (4 ; 2) : $\sqrt{(9^2 + 2^2)} = \sqrt{85} =$ **9,22**
- G à C (5 ; 2) : $\sqrt{(8^2 + 2^2)} = \sqrt{68} =$ **8,25** ← le plus petit
- G à F (8 ; 11) : $\sqrt{(5^2 + 7^2)} = \sqrt{74} =$ **8,60**

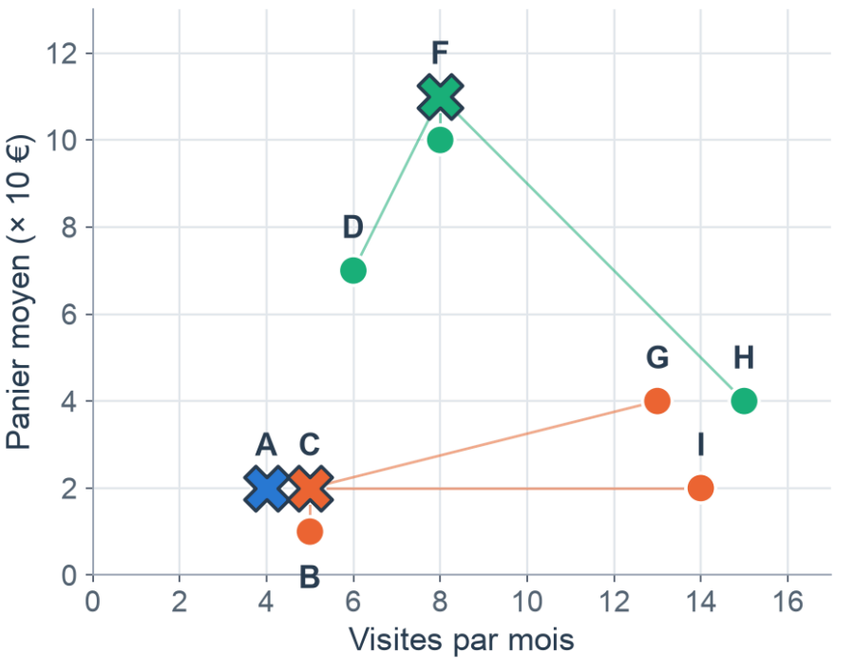
C'est serré (8,25 contre 8,60). On refait ce calcul pour chacun des 9 clients.

Geste 2 : tous les clients

0 Les clients | 1 Les centres | **2 Rattacher** | 3 Déplacer | 2 Rattacher | 3 Déplacer | Fini

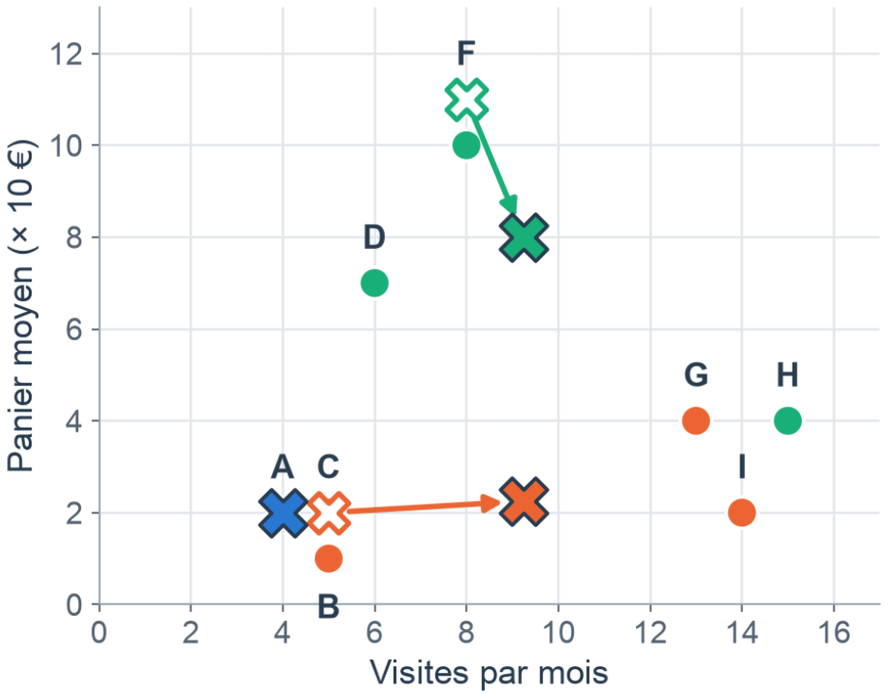
Client	Centre 1	Centre 2	Centre 3	Groupe
A (4 ; 2)	**0**	1,00	9,85	**1**
B (5 ; 1)	1,41	**1,00**	10,44	**2**
C (5 ; 2)	1,00	**0**	9,49	**2**
D (6 ; 7)	5,39	5,10	**4,47**	**3**
E (8 ; 10)	8,94	8,54	**1,00**	**3**
F (8 ; 11)	9,85	9,49	**0**	**3**
G (13 ; 4)	9,22	**8,25**	8,60	**2**
H (15 ; 4)	11,18	10,20	**9,90**	**3**
I (14 ; 2)	10,00	**9,00**	10,82	**2**

la plus petite distance de chaque ligne est entourée



🧮 Geste 3 : déplacer les centres

0 Les clients | 1 Les centres | 2 Rattacher | **3 Déplacer** | 2 Rattacher | 3 Déplacer | Fini



Chaque centre va à la **moyenne** de ses clients (la moyenne est le point « au milieu » du groupe, le plus proche de tous à la fois) :

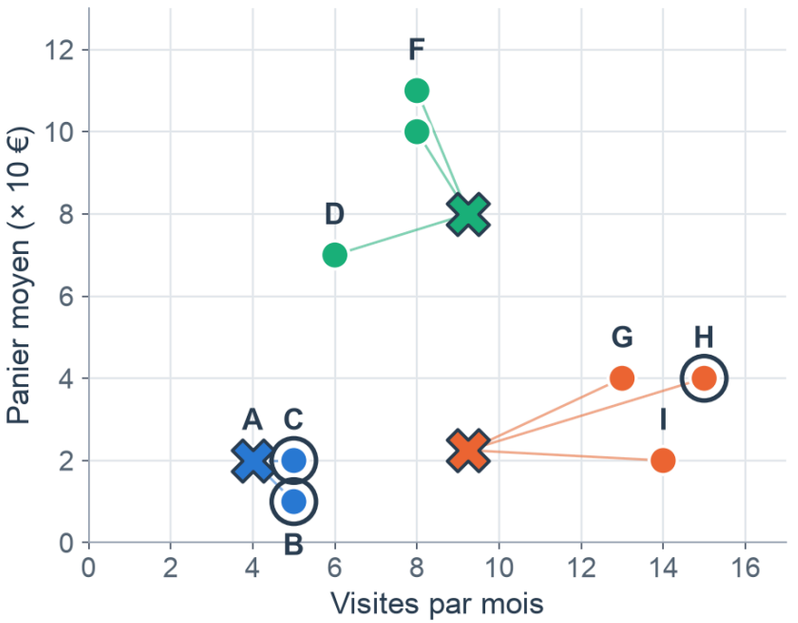
- A seul : **(4 ; 2)**
- B, C, G, I : ((5+5+13+14)/4 ; (1+2+4+2)/4) = **(9,25 ; 2,25)**
- D, E, F, H : ((6+8+8+15)/4 ; (7+10+11+4)/4) = **(9,25 ; 8)**

Deuxième tour

0 Les clients | 1 Les centres | 2 Rattacher | 3 Déplacer | **2 Rattacher** | 3 Déplacer | Fini

Client	Centre 1	Centre 2	Centre 3	Groupe
A (4 ; 2)	**0**	5,26	7,97	**1**
B (5 ; 1)	**1,41**	4,43	8,19	**1**
C (5 ; 2)	**1,00**	4,26	7,35	**1**
D (6 ; 7)	5,39	5,76	**3,40**	**3**
E (8 ; 10)	8,94	7,85	**2,36**	**3**
F (8 ; 11)	9,85	8,84	**3,25**	**3**
G (13 ; 4)	9,22	**4,14**	5,48	**2**
H (15 ; 4)	11,18	**6,01**	7,00	**2**
I (14 ; 2)	10,00	**4,76**	7,65	**2**

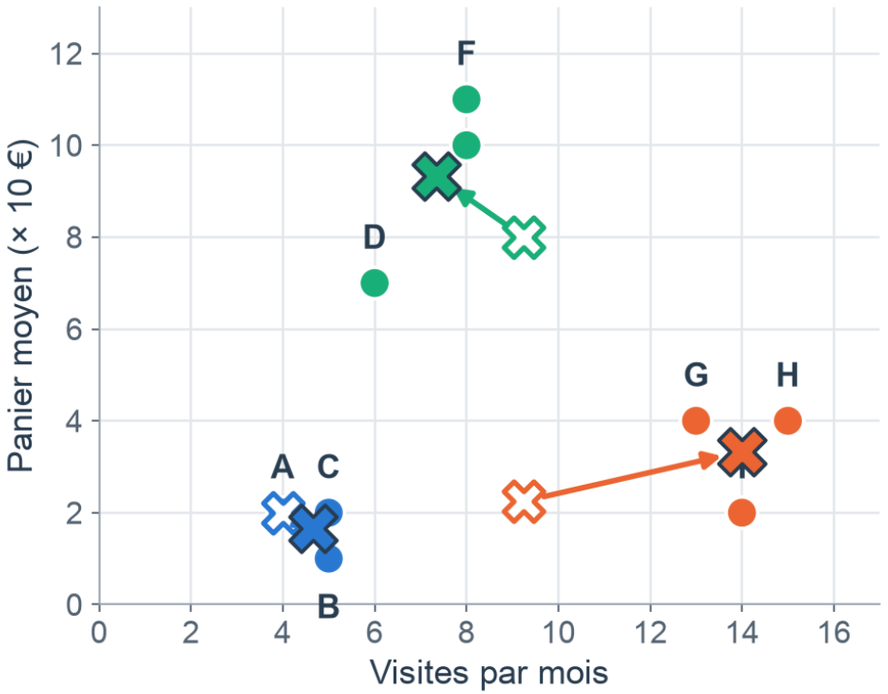
cases cerclées : le client a changé de groupe



B, C et H changent de groupe : les centres déplacés ont corrigé le mauvais tirage.

🧮 Déplacer encore

0 Les clients | 1 Les centres | 2 Rattacher | 3 Déplacer | 2 Rattacher | **3 Déplacer** | Fini



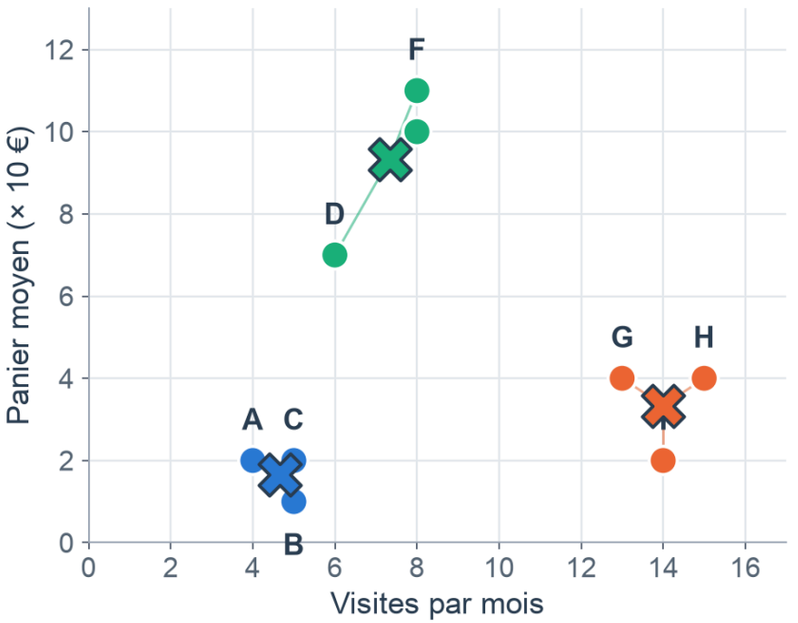
Nouvelles moyennes :

- A, B, C : **(4,67 ; 1,67)**
- G, H, I : **(14 ; 3,33)**
- D, E, F : **(7,33 ; 9,33)**

Troisième tour : plus rien ne bouge

0 Les clients | 1 Les centres | 2 Rattacher | 3 Déplacer | 2 Rattacher | 3 Déplacer | **Fini**

Client	Centre 1	Centre 2	Centre 3	Groupe
A (4 ; 2)	**0,75**	10,09	8,06	**1**
B (5 ; 1)	**0,75**	9,30	8,65	**1**
C (5 ; 2)	**0,47**	9,10	7,70	**1**
D (6 ; 7)	5,50	8,80	**2,69**	**3**
E (8 ; 10)	8,98	8,97	**0,94**	**3**
F (8 ; 11)	9,91	9,74	**1,80**	**3**
G (13 ; 4)	8,65	**1,20**	7,78	**2**
H (15 ; 4)	10,59	**1,20**	9,34	**2**
I (14 ; 2)	9,34	**1,33**	9,91	**2**



Aucun client ne change de groupe : l'algorithme s'arrête. Trois groupes en trois tours.

Que veulent dire les trois groupes ?

A, B, C

4 à 5 visites par mois, **petits paniers** (10 à 20 €).

D, E, F

6 à 8 visites, **gros paniers** (70 à 110 €).

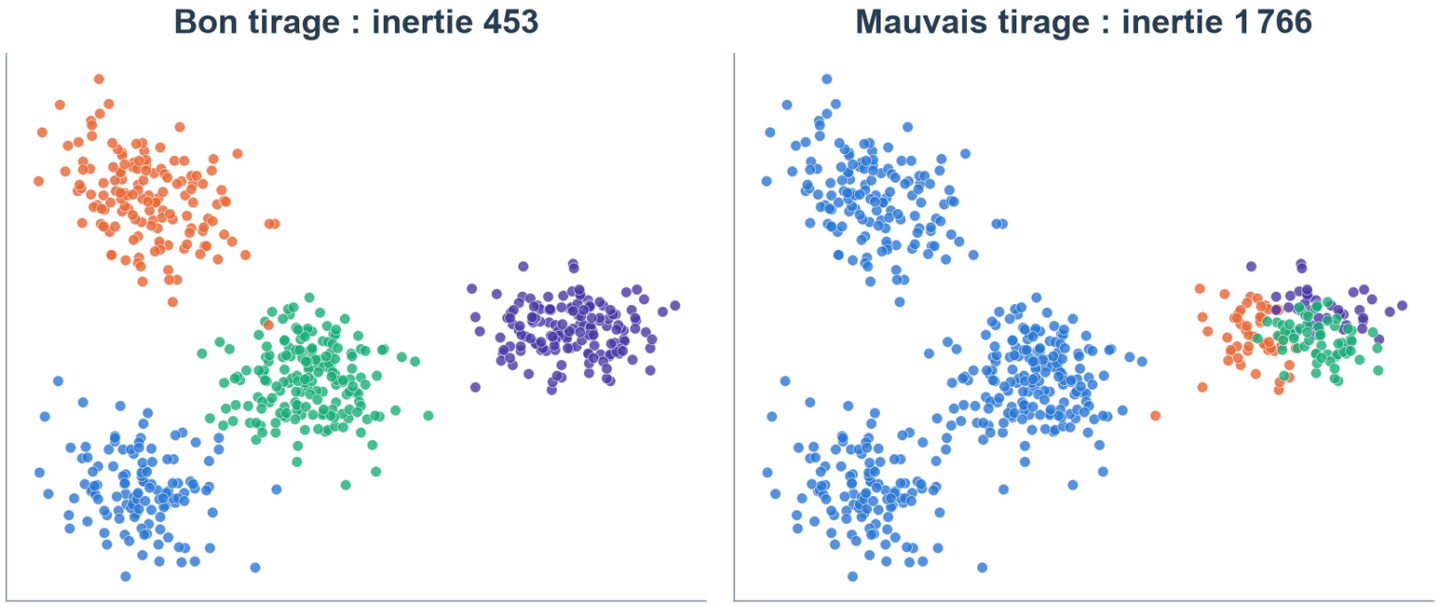
G, H, I

13 à 15 visites, paniers moyens (20 à 40 €).

À nous de **nommer** : étudiants, familles, seniors fidèles ? L'algorithme ne sait pas ce que veut dire « famille » : il a seulement rapproché des chiffres.

Un outil de mesure : l'**inertie**, la somme des distances au carré entre chaque client et son centre : ici **17,33.** Plus elle est petite, plus les groupes sont serrés.

🎲 Et si le tirage est mauvais ?



Même méthode, **autres centres de départ** : sur 600 clients, environ une fois sur trois le résultat est moins bon. Ici, le pire de 60 essais : inertie 1 766 au lieu de 453.

On **relance plusieurs fois** et on garde le meilleur. Python le fait seul : `n_init=10`.

✏️ À vous de jouer

Quelles villes se ressemblent ? Température moyenne et pluie annuelle.

Ville	Température (°C)	Précipitations (mm)
Paris	12	600
Nice	16	700
Marseille	15	500
Lille	11	700

Calculez la distance entre **Nice et Lille**, puis entre **Paris et Lille**. Qui est le plus proche de Lille ?

Tel quel :

Nice – Lille : $\sqrt{(5^2 + 0^2)} = \mathbf{5}$

Paris – Lille : $\sqrt{(1^2 + 100^2)} = \mathbf{100}$

Nice, au soleil, serait la jumelle de Lille ?

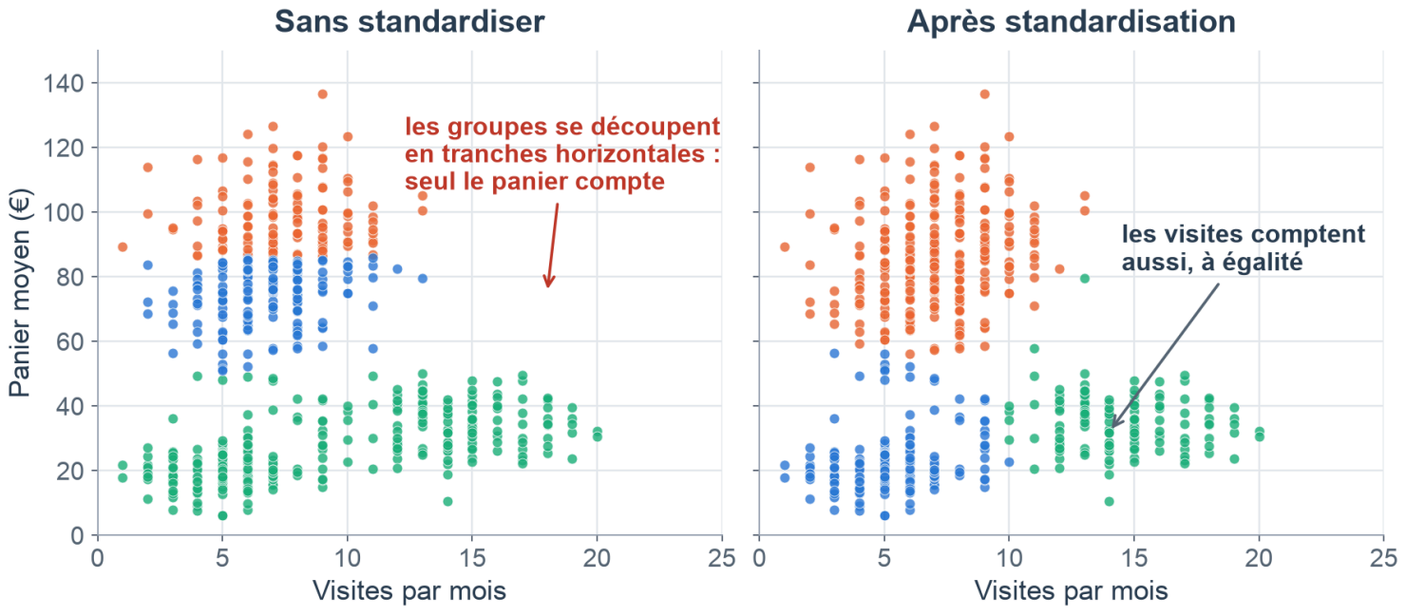
Après standardisation (z-score, séance 1) :

Nice – Lille : **2,43**

Paris – Lille : **1,30**

Paris est le plus proche, comme sur la carte.

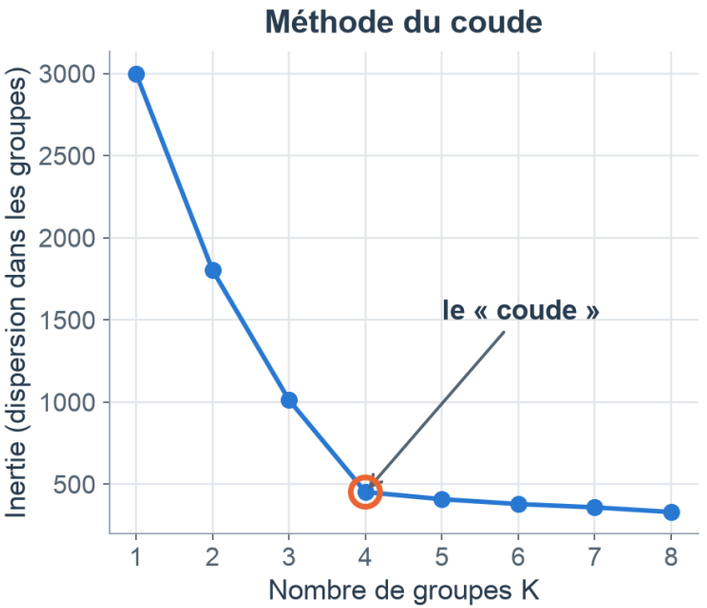
⚖️ Standardiser avant de regrouper

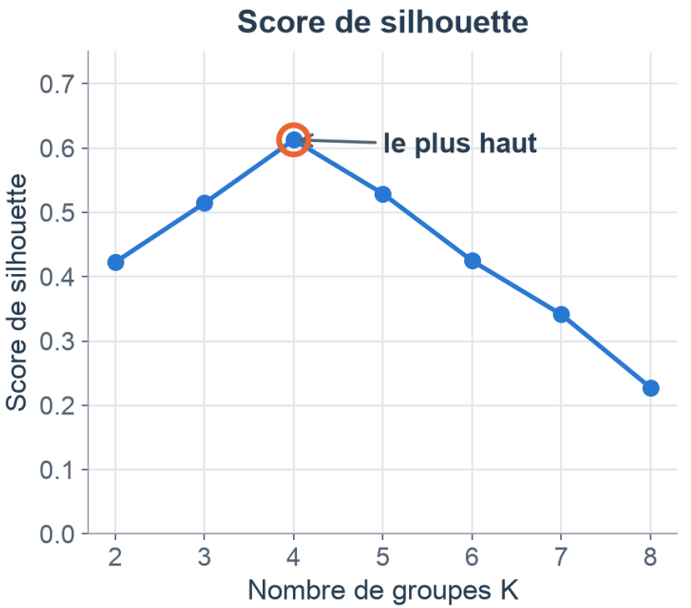


Le panier (en euros) varie bien plus que les visites (écart-type 32 contre 4). Sans standardisation, **seul le panier compte** dans la distance.

Rappel de la séance 1 : on **standardise** chaque colonne. Toutes pèsent alors **autant.** Le clustering se fait toujours sur des données standardisées.

🎯 Combien de groupes ?

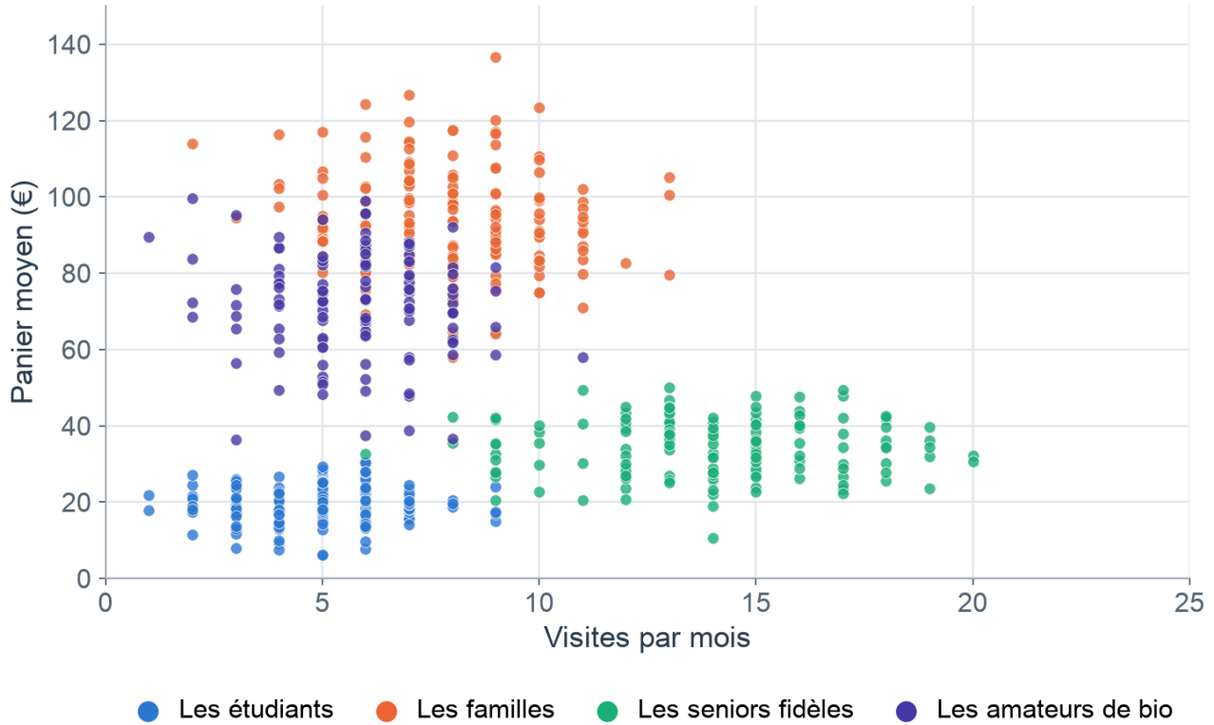




Le coude. L'inertie baisse quand K augmente. Elle chute jusqu'à K = 4 (1 013 puis 453), puis presque plus (409 à K = 5).

La silhouette note, de –1 à +1, à quel point chaque client est **plus proche de son groupe que du groupe voisin.** Son score est le plus haut à K = 4 : **0,61.**

Nos 600 clients en 4 groupes



K-Means avec K = 4 : groupes de **156**, **180**, **144** et **120** clients. Les amateurs de bio et les familles se mêlent sur ces deux axes : ils se distinguent par d'autres colonnes.

🔍 Lire les quatre groupes

Groupe (nombre de clients)	Âge (ans)	Visites par mois	Panier moyen (€)	Part en promo (%)	Part de bio (%)
Les étudiants (156)	23	5	19	55	8
Les familles (180)	40	8	93	30	14
Les seniors fidèles (144)	67	14	34	20	21
Les amateurs de bio (120)	43	6	72	6	63

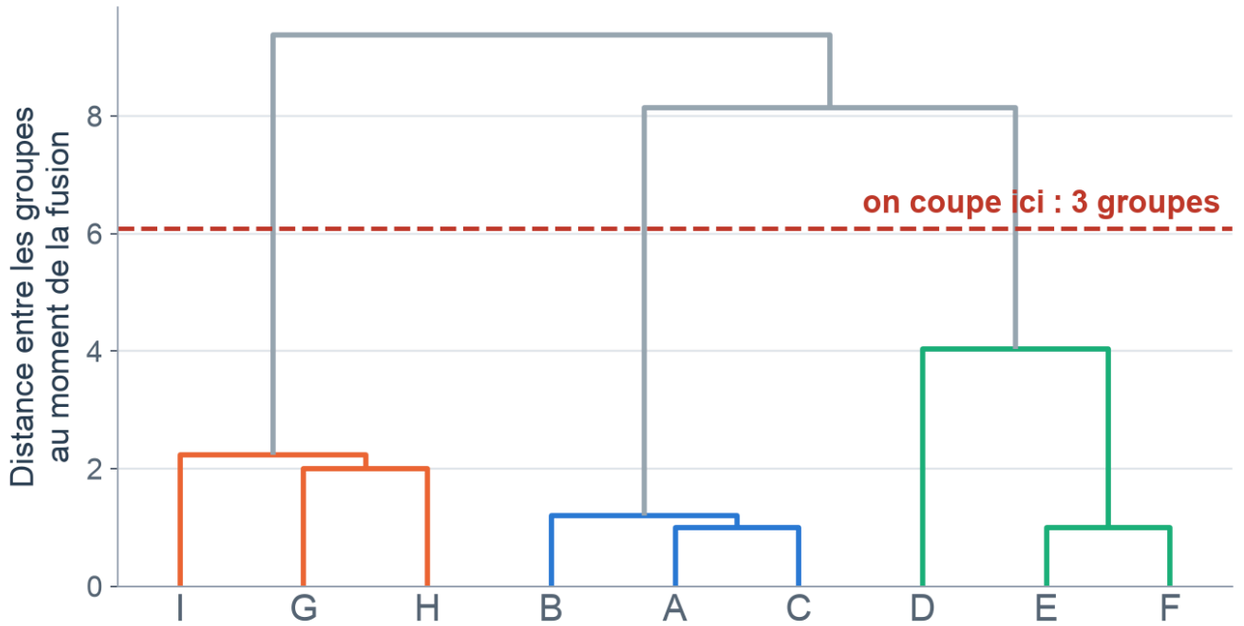
orange : plus que les autres groupes bleu : moins que les autres groupes

- **Étudiants** : promos, petits prix.
- **Familles** : gros paniers, offres en lot.

- **Seniors fidèles** : venues fréquentes, avantages fidélité.
- **Amateurs de bio** : gamme bio, produits premium.

Les noms et les actions viennent de **nous** : lire les groupes est le vrai travail.

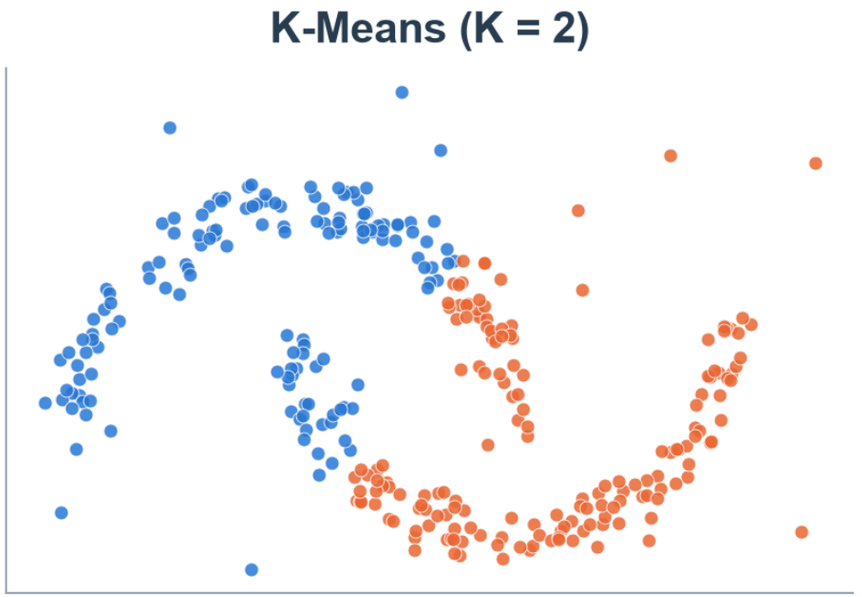
🔗 Trois autres outils (1/3) : hiérarchique



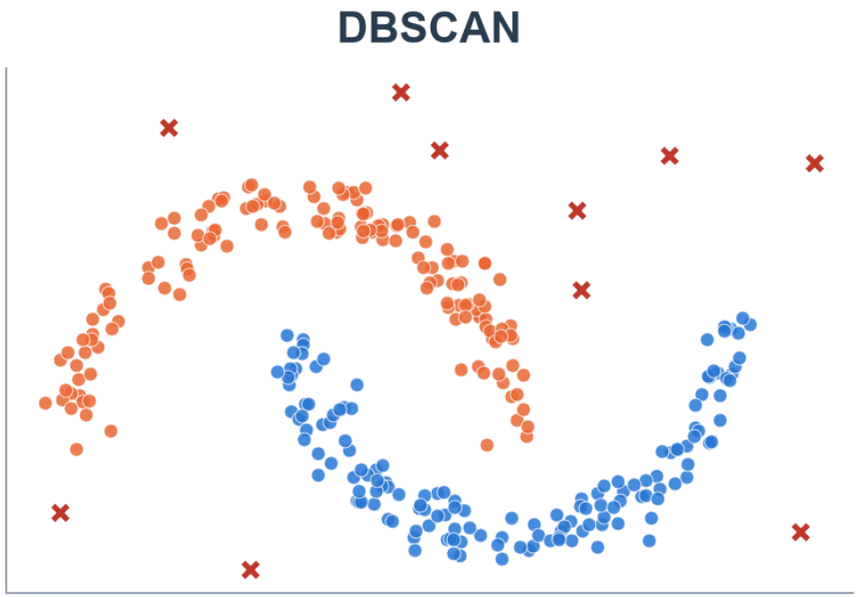
On **fusionne pas à pas** les deux groupes les plus proches, jusqu'à n'en avoir qu'un. Le résultat est un **arbre**.

On **coupe** l'arbre où l'on veut : ici, 3 groupes, **les mêmes que K-Means.** Pas besoin de choisir K avant.

🎯 Trois autres outils (2/3) : DBSCAN



coupe les croissants en deux

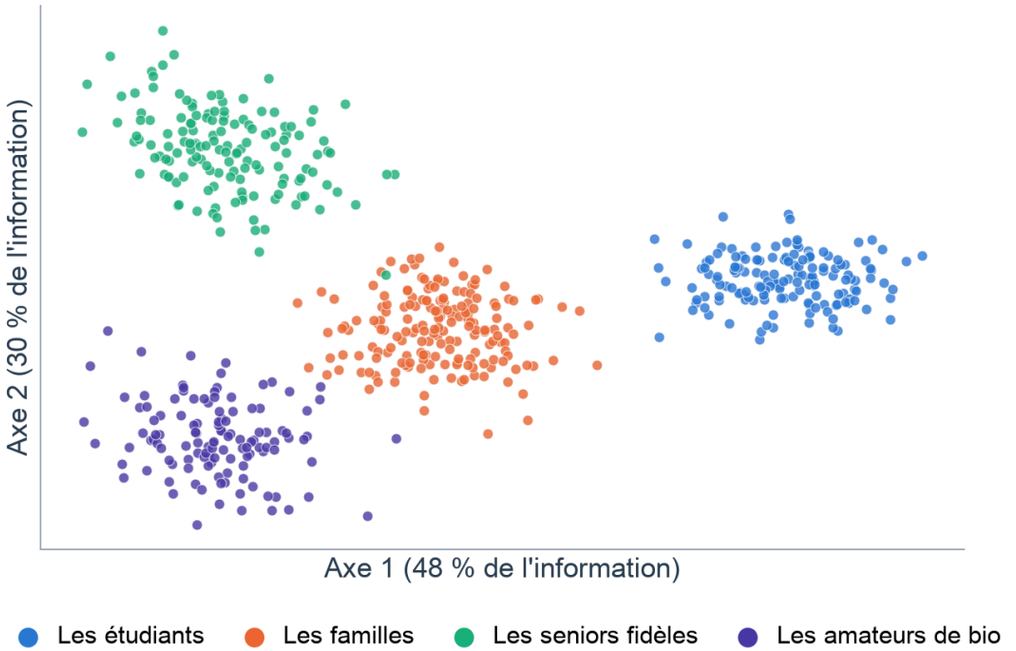


suit chaque croissant, isole le bruit (×)

K-Means cherche des groupes **en boule** : sur deux croissants, il se trompe.

DBSCAN cherche des **zones denses** : il suit **toutes les formes** et met à part les points isolés (le **bruit**, les ×).

Trois autres outils (3/3) : la PCA



5 colonnes ne se dessinent pas. La **PCA** choisit les **axes où les clients s'étalent le plus** et projette dessus : 2 axes gardent **78%** de l'information.

Elle sert à dessiner, pas à trouver les groupes.

⚖️ En un coup d'œil

Outil	À retenir	Réglage à choisir
K-Means	Rapide, simple ; groupes en boule	Le nombre de groupes K
Hiérarchique	Un arbre des ressemblances	Où couper l'arbre
DBSCAN	Toutes les formes, et le bruit isolé	Le rayon ε
PCA	Dessiner plusieurs colonnes sur un plan	Le nombre d'axes

> Dans le doute : **regarder les données d'abord**, commencer par K-Means, comparer avec un autre outil.

💻 En Python : K-Means

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

clients = pd.read_csv("donnees/clients-comtoise-market.csv")
X = clients.drop(columns="id_client")      # 5 colonnes, aucune réponse

X_std = StandardScaler().fit_transform(X)  # chaque colonne pèse autant

km = KMeans(n_clusters=4, n_init=10, random_state=42)
km.fit(X_std)

clients["groupe"] = km.labels_             # le groupe de chaque client
print(clients["groupe"].value_counts())    # la taille des groupes
print(clients.groupby("groupe").mean(numeric_only=True))   # leur profil
```

fit cherche les groupes ; **labels_** donne le groupe de chaque client ; **groupby(...).mean()** les décrit.

💻 En Python : choisir K

```python
from sklearn.metrics import silhouette_score

for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_std)
    print(k, round(km.inertia_), round(silhouette_score(X_std, km.labels_), 2))

# 2 1804 0.42
# 3 1013 0.51
# 4  453 0.61   <- le coude, et la meilleure silhouette
# 5  409 0.53
```

On lit deux colonnes : l'inertie (le coude) et la silhouette (le plus haut). Ici, **K = 4**.

💻 En Python : les autres outils

```
from sklearn.cluster import AgglomerativeClustering, DBSCAN
from sklearn.decomposition import PCA

# hiérarchique : on demande 4 groupes
groupes_h = AgglomerativeClustering(n_clusters=4).fit_predict(X_std)

# DBSCAN : un rayon eps, un nombre minimal de voisins
formes = pd.read_csv("donnees/formes.csv")
groupes_d = DBSCAN(eps=0.2, min_samples=5).fit_predict(formes)    # -1 = du bruit

# PCA : 5 colonnes -> 2 axes, pour dessiner
plan = PCA(n_components=2).fit_transform(X_std)
```

Même schéma qu'avec K-Means : **fit_predict** renvoie le groupe de chaque ligne.

⚠️ Les pièges

Oublier de standardiser. La colonne aux grands nombres décide seule.

Choisir K au hasard. Regarder le coude et la silhouette, puis le sens métier.

Imposer des boules à des formes courbes. Regarder les données ; essayer DBSCAN.

Sur-interpréter. Un groupe est un regroupement de chiffres, pas une vérité : on vérifie avec ceux qui connaissent le métier.

🔑 À retenir

1. **Clustering** : trouver des groupes sans réponse connue (non supervisé).
2. **K-Means** : choisir des centres, rattacher, déplacer, recommencer.
3. **Standardiser** d'abord ; relancer plusieurs fois.
4. **Choisir K** : coude et silhouette, puis le métier.
5. Il existe d'autres outils : **hiérarchique** (un arbre), **DBSCAN** (toutes les formes, et le bruit), **PCA** (dessiner sur un plan).
6. Les groupes se **nomment** et s'**interprètent** : c'est notre travail.

🎓 Conclusion

« Le clustering révèle les structures cachées dans les données, comme des constellations dans un ciel étoilé. »

Vous savez maintenant :

- distinguer classer et regrouper ;
- dérouler K-Means à la main, et le lancer en Python ;
- choisir le nombre de groupes, et les lire ;
- savoir que d'autres outils existent : hiérarchique, DBSCAN, PCA.